CSS 增量匹配器：四种变体、Hoare Logic 与 Kani 验证

css-bitvector-compiler

2026 年 10 月 12 日

摘要

本文给出 bit、tri、recursive-tri 与 quad 四种增量 CSS selector 匹配器的统一语义。四种算法共享同一 selector 状态机、DOM arena 和 mutation 模型，只在缓存抽象与失效处理上不同。本文使用 Hoare triple 描述关键优化的正确性条件，并说明 Kani model checking harness 所证明的局部转换定律。

1 共享状态机

令编译后的状态集合为 Q。每个 DOM 节点 n 接收 parent 与 previous-sibling 两个输入通道，并产生物化输出：

$$P_n, S_n, O_n \in \mathbb{B}^{|Q|}, \qquad \mathbb{B} = \{0, 1\}.$$

例如 selector `.a .b > .c` 产生状态 q_0, q_1, q_2：

$$\begin{aligned} q_0(n) &= p_{.\mathtt{a}}(n), \\ q_1(n) &= P_n[q_0] \wedge p_{.\mathtt{b}}(n), \\ q_2(n) &= P_n[q_1] \wedge p_{.\mathtt{c}}(n), \qquad q_2 \in Q_{\text{accept}}. \end{aligned}$$

descendant combinator 对中间状态增加传播：

$$O_n[q_i] = q_i(n) \vee P_n[q_i].$$

child 不传播；adjacent sibling 从 S_n 读取 predecessor state。

节点 dirty 状态构成全序：

$$\mathsf{Clean} < \mathsf{InputChanged} < \mathsf{NodeChanged}.$$

NodeChanged 表示本地事实变化，不能被输入缓存跳过；InputChanged 表示仅 P_n 或 S_n 变化。

2 Bit：物化输出缓存

Bit 缓存 O_n，对 NodeChanged 和 InputChanged 均完整执行节点程序 F_n：

$$\mathsf{step}_{\text{bit}}(n) = \begin{cases} F_n(P_n, S_n), & d_n \in \{\mathsf{NodeChanged}, \mathsf{InputChanged}\}, \\ O_n, & d_n = \mathsf{Clean}. \end{cases}$$

只有当新旧 O_n 不同时，children 的 parent input 与下一个 sibling 的 sibling input 才会被标记为失效。

3 Tri：输入依赖抽象

Tri 为每个输入 bit 保存 requirement：

$$\mathcal{R} = \{\bot, 0, 1\},$$

其中 $\bot$ 表示本次执行未读取。具体化关系为：

$$\gamma(\bot) = \{0,1\}, \qquad \gamma(0) = \{0\}, \qquad \gamma(1) = \{1\}.$$

因此：

$$\text{compatible}(R, I') \iff \forall i,\ I'[i] \in \gamma(R[i]).$$

Tri skip 的 Hoare triple 是：

$$\left\{ \begin{array}{l} O_n = F_n(I) \\ \wedge\ R \text{ 记录了执行} F_n(I) \text{ 时所有实际读取} \\ \wedge\ \text{compatible}(R, I') \end{array} \right\} \quad \mathsf{skip} \quad \left\{ F_n(I') = O_n \right\}.$$

该规则要求严格保持短路求值。例如 $p_{.\mathtt{b}}(n) \wedge P_n[q]$ 在左侧为 false 时不能读取 $P_n[q]$，否则会把无关输入错误记录为依赖并损失优化。

4 Recursive-tri：调度前检查

Tri 在访问节点后决定 skip；recursive-tri 在父输出变化时先检查 child：

$$P_c \neq P'_c \Longrightarrow \begin{cases} \text{不调度} c, & \text{compatible}(R_c, P'_c), \\ d_c \leftarrow \mathsf{InputChanged}, & \text{otherwise.} \end{cases}$$

其 Hoare triple 为：

$$\begin{array}{c} \left\{ d_c = \mathsf{Clean} \wedge \text{compatible}(R_c, P'_c) \right\} \\ \mathsf{do_not_schedule}(c) \\ \left\{ O'_c = O_c \wedge \neg\,\text{invalidate}(\text{desc}(c)) \right\}. \end{array}$$

当前实现采用逐层 admission check。child 不被调度意味着其输出不变，所以失效不会继续进入 descendants。显式 folded subtree requirement 是可选的未来优化，不是当前正确性的前提。

5 Quad：输出函数抽象

Quad 的抽象域为：

$$\mathcal{Q} = \{\mathbf{0}, \mathbf{1}\} \cup \{\pi_i^P \mid 0 \le i < |Q|\} \cup \{\pi_i^S \mid 0 \le i < |Q|\}.$$

物化函数为：

$$\begin{array}{cc} \mathrm{M}(\mathbf{0}, P, S) = 0, & \mathrm{M}(\mathbf{1}, P, S) = 1, \\ \mathrm{M}(\pi_i^P, P, S) = P[i], & \mathrm{M}(\pi_i^S, P, S) = S[i]. \end{array}$$

计算分两阶段：先求 local/predecessor transition r，再组合 descendant 传播 $r \vee \pi_i^P$。由于 $\mathcal{Q}$ 不能表示任意 OR，按当前输入专门化：

$$\text{specializeOr}(r, c, I) = \begin{cases} (r, r = 1), & \text{M}(r, I) = 1, \\ (c, r = 0), & \text{M}(r, I) = 0. \end{cases}$$

令结果和 decision 为 (q, d)，则 OR refinement 的 Hoare triple 是：

$$\left\{(q, d) = \text{specializeOr}(r, c, I) \wedge \text{holds}(d, I')\right\} \quad x \leftarrow \text{M}(q, I') \quad \left\{x = \text{M}(r, I') \vee \text{M}(c, I')\right\}.$$

accept state 必须具体化。若 $(b, d) = \text{specializeConcrete}(q, I)$，则：

$$\left\{\text{holds}(d, I')\right\} \quad x \leftarrow \text{M}(b, I') \quad \left\{x = \text{M}(q, I')\right\}.$$

decision 失效时执行完整重算；decision 仍成立时只重新 materialize abstract output。这样纯传播节点无需重新执行 selector predicates。

6 Mutation 与结构不变量

- class、id、attribute 和派生 pseudo 变化属于 NodeChanged。
- `:hover` 向下派生，`:focus-within` 向上派生。
- sibling 插入/删除使后续节点的 nth position 失效。
- accept bit 只表示当前节点匹配，不能从祖先直接传播。
- attribute shortcut 仅在局部输出与匹配集合均不变时停止传播。

7 Kani proof harness

proof harness 直接调用生产代码中的纯函数。Kani 固定 parent/sibling vector 长度为 2，但 index、所有输入 bit 和 Quad tag 都是符号值。

1. `tri_requirement_reuse_is_sound`：compatible 的 Zero/One requirement 保证被读取 bit 不变。
2. `quad_projection_materializes_current_input`：parent/sibling projection 精确读取对应输入。
3. `specialized_or_refines_boolean_or`：decision 成立时专门化结果等价于原布尔 OR。
4. `concrete_accept_reuse_is_sound`：decision 成立时 concrete accept 可安全复用。

运行：

```
cargo xtask verify
cargo xtask check
cargo xtask corpus
```

Kani 证明覆盖逐 bit 的局部转换定律。端到端测试使用 13 个真实站点，并与独立 naive oracle 做 differential validation。

8 Lean 4 归纳证明

`formal/` 包含由 Lean 内核检查的算法模型证明。证明覆盖任意状态数的 tri 复用、quad 专门化与状态列表组合。选择器证明对任意有限 parent/sibling 上下文归纳，证明状态递推与直接解释器一致。recursive-tri 的 admission 定理要求另一输入通道保持不变。

从项目根目录执行：

```
cd formal
lake build
```

构建固定使用 Lean 4.31.0，并检查项目声明的公理依赖。

定理列表与证明边界见 `formal/README.md`。

森林模型还证明修改失效维护、有限修改序列和 dirty 遍历的正确性与终止性。`formal/rust/` 使用 Charon/Aeneas 从生产 Rust 生成 Lean 定义。源码证明覆盖 quad 局部转换、输入条件检查和 `Dom::mark`。输入条件检查拒绝长度不等的数组，证明包含循环终止和边界安全。标记证明使用 HashMap 接口模型，并要求父节点内部 ID 小于子节点 ID。源码证明还覆盖实例分配、清空、递归删除和前驱兄弟查询。分配保留 ID 新鲜性，清空保留计数器。删除在子 ID 递增的前提下终止，并保留未删除节点的记录。兄弟查询忽略失效槽位。调度器转换检查使用不展开的求值器，因此不构成调度器证明。实际 arena 构造、mutation 与完整调度器的源码精化仍未完成。

9 当前限制

当前不支持 `:has`、general sibling、`:last-*`、`:nth-last-*` 和 selector-list pseudo。CSS parser 与可变 Rust DOM arena 的端到端正确性仍未证明。